

API Endpoints — b-secure Backend

Base URL: <https://api.bsecure.in> **Auth:** Authorization: Bearer <access_token> (except public endpoints) **Content-Type:** application/json

Table of Contents

1. [Authentication Endpoints](#)
 2. [User Endpoints](#)
 3. [Guard Endpoints](#)
 4. [Booking Endpoints](#)
 5. [Tracking Endpoints](#)
 6. [Payment Endpoints](#)
 7. [Notification Endpoints](#)
 8. [SOS Endpoints](#)
 9. [Review Endpoints](#)
 10. [Admin Panel Endpoints](#)
 11. [Webhook Endpoints](#)
 12. [ViewSet Code Reference](#)
-

1. Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/send-otp/	None	Send OTP to phone number
POST	/api/auth/verify-otp/	None	Verify OTP, receive JWT tokens
POST	/api/auth/refresh/	None	Refresh access token
POST	/api/auth/logout/	Bearer	Blacklist refresh token
POST	/api/auth/social/google/	None	Google Sign-In

Method	Endpoint	Auth	Description
POST	/api/auth/social/apple/	None	Apple Sign-In

POST /api/auth/send-otp/

Request:

```
{
  "phone_number": "+919876543210",
  "role": "USER"
}
```

Response 200:

```
{
  "data": {
    "message": "OTP sent successfully",
    "expires_in": 300
  }
}
```

Response 429 (rate limited):

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many OTP requests. Try again in 10 minutes.",
    "details": {}
  }
}
```

POST /api/auth/verify-otp/

Request:

```
{
  "phone_number": "+919876543210",
  "otp_code": "482619",
  "role": "USER"
}
```

Response 200 (existing user):

```
{
  "data": {
    "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

```

        "is_new_user": false,
        "role": "USER",
        "user_id": "550e8400-e29b-41d4-a716-446655440000"
    }
}

```

Response 200 (new user):

```

{
  "data": {
    "access": "eyJ...",
    "refresh": "eyJ...",
    "is_new_user": true,
    "role": "USER",
    "user_id": "660f9511-f30c-52e5-b827-557766551111"
  }
}

```

2. User Endpoints

Method	Endpoint	Auth	Description
GET	/api/users/me/	Bearer	Get own profile
PUT/PATCH	/api/users/me/	Bearer	Update own profile
POST	/api/users/me/ photo/	Bearer	Upload profile photo
GET	/api/users/me/ addresses/	Bearer	List saved addresses
POST	/api/users/me/ addresses/	Bearer	Add new address
PUT	/api/users/me/ addresses/ {id}/	Bearer	Update address
DELETE	/api/users/me/ addresses/ {id}/	Bearer	Delete address
GET	/api/users/me/ emergency- contacts/	Bearer	List emergency contacts
POST	/api/users/me/ emergency- contacts/	Bearer	Add emergency contact
PUT		Bearer	

Method	Endpoint	Auth	Description
	/api/users/me/emergency-contacts/{id}/		Update emergency contact
DELETE	/api/users/me/emergency-contacts/{id}/	Bearer	Delete emergency contact
GET	/api/users/me/bookings/	Bearer	Booking history
GET	/api/users/me/wallet/	Bearer	Wallet balance + recent transactions
PUT	/api/users/me/fcm-token/	Bearer	Update FCM push token
DELETE	/api/users/me/account/	Bearer	Request account deletion

GET /api/users/me/

Response 200:

```
{
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "phone_number": "+919876543210",
    "full_name": "Rahul Sharma",
    "email": "rahul@example.com",
    "gender": "MALE",
    "profile_photo_url": "https://cdn.bsecure.in/users/photos/xxx.jpg",
    "role": "USER",
    "is_suspended": false,
    "wallet_balance": "250.00",
    "total_bookings": 12,
    "created_at": "2026-01-15T08:30:00Z"
  }
}
```

POST /api/users/me/addresses/

Request:

```
{
  "label": "HOME",
  "line1": "42, Indiranagar 1st Cross",
  "line2": "HAL 2nd Stage",
  "city": "Bengaluru",
}
```

```
    "state": "Karnataka",
    "pincode": "560038",
    "latitude": 12.9716,
    "longitude": 77.5946,
    "is_default": true
}
```

Response 201:

```
{
  "data": {
    "id": "abc123...",
    "label": "HOME",
    "line1": "42, Indiranagar 1st Cross",
    "city": "Bengaluru",
    "state": "Karnataka",
    "pincode": "560038",
    "latitude": "12.971600",
    "longitude": "77.594600",
    "is_default": true
  }
}
```

3. Guard Endpoints

Method	Endpoint	Auth	Description
GET	/api/guards/me/	Bearer (Guard)	Guard's own profile
PUT/PATCH	/api/guards/me/	Bearer (Guard)	Update guard profile
GET	/api/guards/me/documents/	Bearer (Guard)	List uploaded documents
POST	/api/guards/me/documents/	Bearer (Guard)	Upload a document
GET	/api/guards/me/availability/	Bearer (Guard)	Get availability schedule
PUT	/api/guards/me/availability/	Bearer (Guard)	Update availability schedule
PUT	/api/guards/me/online-status/	Bearer (Guard)	Toggle online/offline
GET		Bearer (Guard)	

Method	Endpoint	Auth	Description
	/api/guards/me/booking-requests/		Pending booking requests
POST	/api/guards/me/booking-requests/{id}/accept/	Bearer (Guard)	Accept a booking
POST	/api/guards/me/booking-requests/{id}/decline/	Bearer (Guard)	Decline a booking
GET	/api/guards/me/earnings/	Bearer (Guard)	Earnings summary
GET	/api/guards/me/payouts/	Bearer (Guard)	Payout history
GET	/api/guards/{id}/	Bearer	Public guard profile
GET	/api/guards/{id}/reviews/	Bearer	Guard's public reviews
GET	/api/guards/nearby/	Bearer	Nearby available guards (internal, called by booking service)

PUT /api/guards/me/online-status/

Request:

```
{
  "is_online": true,
  "latitude": 12.9716,
  "longitude": 77.5946
}
```

Response 200:

```
{
  "data": {
    "is_online": true,
    "message": "You are now online and visible to users.",
    "active_booking_requests": 0
  }
}
```

POST /api/guards/me/documents/

Request: multipart/form-data

document_type: POLICE_CERT
file: <binary file>
expiry_date: 2027-03-15 (optional)

Response 201:

```
{
  "data": {
    "id": "doc-uuid",
    "document_type": "POLICE_CERT",
    "document_type_display": "Police Verification Certificate",
    "status": "UPLOADED",
    "expiry_date": "2027-03-15",
    "uploaded_at": "2026-05-28T10:00:00Z",
    "file_url": null
  }
}
```

file_url is null until admin approves. After approval, a pre-signed S3 URL is returned (15-min expiry).

GET /api/guards/me/earnings/

Query params: ?period=this_week|this_month|last_month|custom&from=2026-05-01&to=2026-05-28

Response 200:

```
{
  "data": {
    "period": "this_week",
    "total_earnings": "3200.00",
    "total_sessions": 8,
    "total_hours": 48.5,
    "pending_payout": "3200.00",
    "breakdown": [
      {
        "booking_id": "uuid",
        "date": "2026-05-27",
        "duration_hours": 6,
        "earnings": "450.00",
        "service_type": "HOURLY"
      }
    ]
  }
}
```

4. Booking Endpoints

Method	Endpoint	Auth	Description
POST	/api/bookings/	Bearer (User)	Create new booking
GET	/api/bookings/{id}/	Bearer	Booking detail
POST	/api/bookings/{id}/cancel/	Bearer	Cancel booking
POST	/api/bookings/{id}/generate-start-otp/	Bearer (User)	Generate session start OTP
POST	/api/bookings/{id}/verify-start-otp/	Bearer (Guard)	Verify OTP to start session
POST	/api/bookings/{id}/generate-end-otp/	Bearer (User)	Generate session end OTP
POST	/api/bookings/{id}/verify-end-otp/	Bearer (Guard)	Verify OTP to end session
POST	/api/bookings/{id}/guard-arrived/	Bearer (Guard)	Guard marks arrival
POST	/api/bookings/{id}/guard-en-route/	Bearer (Guard)	Guard starts travel
POST	/api/bookings/{id}/checkin/	Bearer (Guard)	Guard check-in during active session
GET	/api/bookings/{id}/checkins/	Bearer	Check-in history for session
GET	/api/bookings/active/	Bearer	Current active booking
POST	/api/bookings/{id}/dispute/	Bearer	Raise a dispute on completed session

POST /api/bookings/

Request:


```

{
  "service_type": "HOURLY",
  "guard_type_requested": "UNARMED",
  "scheduled_start": "2026-05-28T20:00:00+05:30",
  "scheduled_end": "2026-05-28T23:00:00+05:30",
  "is_immediate": false,
  "service_latitude": 12.9716,
  "service_longitude": 77.5946,
  "service_address": "42, Indiranagar 1st Cross, Bengaluru"
}

```

Response 201:

```

{
  "data": {
    "id": "booking-uuid",
    "status": "REQUESTED",
    "service_type": "HOURLY",
    "guard_type_requested": "UNARMED",
    "scheduled_start": "2026-05-28T14:30:00Z",
    "scheduled_end": "2026-05-28T17:30:00Z",
    "service_address": "42, Indiranagar 1st Cross, Bengaluru",
    "estimated_amount": "450.00",
    "estimated_guard_arrival_minutes": null,
    "guard": null,
    "created_at": "2026-05-28T10:05:00Z"
  }
}

```

Errors:

// Insufficient wallet balance

```

{
  "error": {
    "code": "INSUFFICIENT_BALANCE",
    "message":
      "Your wallet balance (₹150) is insufficient for this
      booking (₹450). Please top up."
  }
}

```

// No guards available

```

{
  "error": {
    "code": "NO_GUARDS_AVAILABLE",
    "message": "No verified guards available in your area
      right now. Try scheduling for later."
  }
}

```

GET /api/bookings/{id}/

Response 200:

```
{
  "data": {
    "id": "booking-uuid",
    "status": "ACTIVE",
    "service_type": "HOURLY",
    "guard_type_requested": "UNARMED",
    "scheduled_start": "2026-05-28T14:30:00Z",
    "scheduled_end": "2026-05-28T17:30:00Z",
    "session_started_at": "2026-05-28T14:35:00Z",
    "session_ended_at": null,
    "service_address": "42, Indiranagar 1st Cross, Bengaluru",
    "service_latitude": "12.971600",
    "service_longitude": "77.594600",
    "total_amount": "450.00",
    "platform_fee": "67.50",
    "guard_earnings": "382.50",
    "guard": {
      "id": "guard-uuid",
      "name": "Suresh Kumar",
      "phone_number_masked": "+91****3210",
      "profile_photo_url": "https://cdn.bsecure.in/...",
      "average_rating": "4.8",
      "total_sessions": 142,
      "guard_type": "UNARMED",
      "current_latitude": 12.9720,
      "current_longitude": 77.5950
    },
    "user": {
      "id": "user-uuid",
      "name": "Rahul Sharma",
      "phone_number_masked": "+91****9210"
    }
  }
}
```

5. Tracking Endpoints

WebSocket endpoints are documented in `realtime.md`. REST endpoints below:

Method	Endpoint	Auth	Description
GET	/api/tracking/sessions/	Bearer	Full location replay for a session

Method	Endpoint	Auth	Description
GET	{booking_id}/ history/ /api/tracking/ sessions/ {booking_id}/ current/	Bearer	Latest guard location for active session

GET /api/tracking/sessions/{booking_id}/current/

Response 200:

```
{
  "data": {
    "booking_id": "booking-uuid",
    "guard_id": "guard-uuid",
    "latitude": 12.9720,
    "longitude": 77.5950,
    "accuracy_meters": 4.5,
    "speed_kmh": 0.0,
    "timestamp": "2026-05-28T14:37:22Z",
    "status": "ACTIVE",
    "eta_seconds": null
  }
}
```

GET /api/tracking/sessions/{booking_id}/history/

Query params: ?

from=2026-05-28T14:00:00Z&to=2026-05-28T17:00:00Z&interval=10
(interval in seconds, for downsampling)

Response 200:

```
{
  "data": {
    "booking_id": "booking-uuid",
    "total_points": 2160,
    "duration_seconds": 10800,
    "points": [
      {
        "lat": 12.9716,
        "lng": 77.5946,
        "ts": "2026-05-28T14:35:00Z",
        "spd": 0.0
      },
      {
        "lat": 12.9718,
        "lng": 77.5948,
```

```
    "ts": "2026-05-28T14:35:05Z",  
    "spd": 3.2  
  }  
]  
}
```

6. Payment Endpoints

Method	Endpoint	Auth	Description
GET	/api/payments/wallet/	Bearer (User)	Wallet balance + transaction list
POST	/api/payments/wallet/topup/initiate/	Bearer (User)	Create Razorpay/Stripe order
POST	/api/payments/wallet/topup/confirm/	Bearer (User)	Confirm payment (client-side verify)
GET	/api/payments/transactions/	Bearer (User)	Paginated transaction history
GET	/api/payments/invoices/{booking_id}/	Bearer	Download invoice PDF
POST	/api/payments/webhook/razorpay/	None (HMAC)	Razorpay webhook handler
POST	/api/payments/webhook/stripe/	None (Sig)	Stripe webhook handler

POST /api/payments/wallet/topup/initiate/

Request:

```
{  
  "amount": 500,  
  "gateway": "RAZORPAY"  
}
```

Response 200:

```
{  
  "data": {
```

```
    "order_id": "order_uuid_internal",
    "gateway_order_id": "order_OFWbmMi5aNMfR9",
    "gateway": "RAZORPAY",
    "amount": 500,
    "currency": "INR",
    "razorpay_key_id": "rzp_live_xxxxxxxxx"
  }
}
```

Client uses gateway_order_id + razorpay_key_id to open Razorpay checkout SDK.

POST /api/payments/wallet/topup/confirm/

Request:

```
{
  "gateway_order_id": "order_OFWbmMi5aNMfR9",
  "gateway_payment_id": "pay_OFXc3hzK8fx0km",
  "gateway_signature": "abc123signaturestring"
}
```

Response 200:

```
{
  "data": {
    "success": true,
    "new_balance": "650.00",
    "transaction_id": "txn-uuid"
  }
}
```

7. Notification Endpoints

Method	Endpoint	Auth	Description
GET	/api/notifications/	Bearer	In-app notification inbox
POST	/api/notifications/{id}/read/	Bearer	Mark notification as read
POST	/api/notifications/read-all/	Bearer	Mark all as read
GET		Bearer	Unread count badge

Method	Endpoint	Auth	Description
GET	/api/notifications/unread-count/	Bearer	Get preferences
	/api/users/me/notification-preferences/		
PUT	/api/users/me/notification-preferences/	Bearer	Update preferences

GET /api/notifications/

Query params: ?page=1&page_size=20

Response 200:

```
{
  "data": [
    {
      "id": "notif-uuid",
      "notification_type": "GUARD_ASSIGNED",
      "title": "Guard Assigned",
      "body": "Suresh Kumar is on the way to your location.
ETA: 8 minutes.",
      "data": {
        "booking_id": "booking-uuid",
        "guard_id": "guard-uuid"
      },
      "is_read": false,
      "created_at": "2026-05-28T14:30:00Z"
    }
  ],
  "count": 45,
  "next": "/api/notifications/?page=2",
  "previous": null
}
```

8. SOS Endpoints

Method	Endpoint	Auth	Description
POST	/api/sos/trigger/	Bearer	Trigger SOS alert
GET	/api/sos/alerts/	Bearer (User)	User's SOS history

Method	Endpoint	Auth	Description
GET	/api/sos/alerts/{id}/	Bearer	SOS alert detail
POST	/api/incidents/	Bearer	File an incident report
GET	/api/incidents/	Bearer (User)	User's incident history
GET	/api/incidents/{id}/	Bearer	Incident detail
POST	/api/incidents/{id}/evidence/	Bearer	Upload evidence file

POST /api/sos/trigger/

Request:

```
{
  "trigger_method": "BUTTON",
  "latitude": 12.9716,
  "longitude": 77.5946,
  "booking_id": "booking-uuid"
}
```

Response 200:

```
{
  "data": {
    "sos_id": "sos-uuid",
    "status": "TRIGGERED",
    "message": "SOS alert sent. Emergency contacts have been notified. Our control room has been alerted.",
    "emergency_contacts_notified": 2,
    "triggered_at": "2026-05-28T14:40:00Z"
  }
}
```

This endpoint must respond in < **1 second**. SOS record is written synchronously. Emergency contact alerts are queued to Celery.

POST /api/incidents/

Request: multipart/form-data

booking_id: booking-uuid
incident_type: GUARD_MISCONDUCT

severity: HIGH
description: The guard used inappropriate language and made threatening gestures.
evidence_files: <file1>, <file2>

Response 201:

```
{
  "data": {
    "incident_id": "incident-uuid",
    "status": "OPEN",
    "message": "Incident report filed. Our team will review within 24 hours.",
    "booking_id": "booking-uuid",
    "incident_type": "GUARD_MISCONDUCT",
    "evidence_count": 2
  }
}
```

9. Review Endpoints

Method	Endpoint	Auth	Description
POST	/api/reviews/	Bearer (User)	Submit post-session review
GET	/api/guards/{id}/reviews/	Bearer	Get guard's public reviews
POST	/api/reviews/{id}/flag/	Bearer	Flag a review

POST /api/reviews/

Request:

```
{
  "booking_id": "booking-uuid",
  "overall_rating": 5,
  "punctuality_rating": 5,
  "professionalism_rating": 4,
  "communication_rating": 5,
  "alertness_rating": 5,
  "comment": "Suresh was punctual, professional, and made me feel very safe. Highly recommend!"
}
```

Response 201:


```

{
  "data": {
    "review_id": "review-uuid",
    "overall_rating": 5,
    "guard_new_average": "4.82",
    "message": "Thank you for your review!"
  }
}

```

10. Admin Panel Endpoints

All admin endpoints require IsAdminUser permission (user must have is_staff=True). **Prefix:** /api/admin/

Method	Endpoint	Description
GET	/api/admin/dashboard/stats/	KPI summary (real-time)
GET	/api/admin/dashboard/live-map/	All active sessions + guard locations
GET	/api/admin/users/	Paginated user list with filters
GET	/api/admin/users/{id}/	User detail with full history
POST	/api/admin/users/{id}/suspend/	Suspend user
POST	/api/admin/users/{id}/unsuspend/	Unsuspend user
POST	/api/admin/users/{id}/ban/	Permanently ban user
POST	/api/admin/users/{id}/credit-wallet/	Admin wallet credit
GET	/api/admin/guards/	Paginated guard list with filters
GET	/api/admin/guards/{id}/	Guard detail
POST	/api/admin/guards/{id}/approve/	Approve guard (all docs verified)
POST	/api/admin/guards/{id}/suspend/	Suspend guard
POST		Approve single document

Method	Endpoint	Description
POST	/api/admin/guards/{id}/documents/{doc_id}/approve/ /api/admin/guards/{id}/documents/{doc_id}/reject/	Reject document with notes
GET	/api/admin/guards/verification-queue/	Guards pending document review
GET	/api/admin/bookings/	Paginated bookings with filters
GET	/api/admin/bookings/{id}/	Booking detail with session replay data
POST	/api/admin/bookings/{id}/reassign/	Reassign guard
POST	/api/admin/bookings/{id}/cancel/	Admin cancel booking
GET	/api/admin/sos/alerts/	SOS alerts (filter by status)
GET	/api/admin/sos/alerts/{id}/	SOS detail
POST	/api/admin/sos/alerts/{id}/acknowledge/	Acknowledge SOS
POST	/api/admin/sos/alerts/{id}/resolve/	Resolve SOS with notes
GET	/api/admin/incidents/	Incident list
POST	/api/admin/incidents/{id}/assign/	Assign incident to staff
POST	/api/admin/incidents/{id}/resolve/	Resolve incident
GET	/api/admin/payments/revenue/	Revenue report
GET	/api/admin/payments/payouts/	Guard payout queue
POST	/api/admin/payments/payouts/{id}/approve/	Approve payout

Method	Endpoint	Description
POST	/api/admin/payments/refunds/{booking_id}/	Issue refund
GET	/api/admin/analytics/bookings/	Booking analytics
GET	/api/admin/analytics/guards/	Guard performance analytics
GET	/api/admin/analytics/revenue/	Revenue analytics
POST	/api/admin/notifications/broadcast/	Broadcast push notification

GET /api/admin/dashboard/stats/

Response 200:

```
{
  "data": {
    "realtime": {
      "active_sessions": 47,
      "guards_online": 183,
      "users_in_app": 312,
      "open_sos_alerts": 1,
      "pending_guard_approvals": 8
    },
    "today": {
      "total_bookings": 143,
      "completed_bookings": 98,
      "cancelled_bookings": 12,
      "gross_revenue": "52400.00",
      "new_users": 34,
      "new_guards": 5
    },
    "this_month": {
      "total_bookings": 3412,
      "gross_revenue": "1284500.00"
    }
  }
}
```

POST /api/admin/guards/{id}/documents/{doc_id}/reject/

Request:

```
{
  "review_notes": "The police verification certificate is more
    than 1 year old. Please upload a fresh certificate dated
    within the last 6 months."
}
```

Response 200:

```
{
  "data": {
    "document_id": "doc-uuid",
    "status": "REJECTED",
    "review_notes": "The police verification certificate is
      more than 1 year old...",
    "guard_notified": true
  }
}
```

11. Webhook Endpoints

These endpoints receive callbacks from payment gateways. They must be: -
Public (no JWT auth) - Verified via HMAC signature before processing -
Idempotent (safe to receive same event twice)

POST /api/payments/webhook/razorpay/

Handles: payment.captured, payment.failed, payout.processed,
 payout.failed

apps/payments/webhooks.py

```
import hmac, hashlib
from django.conf import settings
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import AllowAny
from rest_framework.response import Response
from .tasks import process_razorpay_event
```

```
@api_view(['POST'])
@permission_classes([AllowAny])
def razorpay_webhook(request):
    # Step 1: Verify HMAC signature
    signature = request.headers.get('X-Razorpay-Signature', '')
    payload = request.body
    secret = settings.RAZORPAY_WEBHOOK_SECRET.encode()
    expected = hmac.new(secret, payload,
        hashlib.sha256).hexdigest()
```

```

if not hmac.compare_digest(expected, signature):
    return Response({'error': 'Invalid signature'},
                    status=400)

# Step 2: Hand off to Celery (async, idempotent)
event = request.data
process_razorpay_event.apply_async(
    args=[event],
    queue='high_priority'
)

# Step 3: Immediately return 200 to gateway
return Response({'status': 'received'}, status=200)

```

12. ViewSet Code Reference

BookingViewSet

```

# apps/bookings/views.py

from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from django_fsm import TransitionNotAllowed

from .models import Booking
from .serializers import (
    BookingCreateSerializer, BookingDetailSerializer,
    BookingCancelSerializer
)
from .services import BookingService
from utils.permissions import IsVerifiedUser, IsBookingParticipant

class BookingViewSet(viewsets.GenericViewSet):
    permission_classes = [IsAuthenticated, IsVerifiedUser]

    def get_queryset(self):
        user = self.request.user
        if hasattr(user, 'guard_profile'):
            return
            Booking.objects.filter(guard=user.guard_profile)
        return Booking.objects.filter(user=user)

    def get_serializer_class(self):
        if self.action == 'create':
            return BookingCreateSerializer
        return BookingDetailSerializer

```

```

def create(self, request):
    """POST /api/bookings/ - Create new booking."""
    serializer = BookingCreateSerializer(data=request.data)
    serializer.is_valid(raise_exception=True)
    try:
        booking = BookingService.create_booking(
            user=request.user,
            data=serializer.validated_data
        )
        return Response(
            {'data': BookingDetailSerializer(booking).data},
            status=status.HTTP_201_CREATED
        )
    except BookingService.InsufficientBalanceError as e:
        return Response(
            {'error': {'code': 'INSUFFICIENT_BALANCE',
                'message': str(e)}},
            status=status.HTTP_400_BAD_REQUEST
        )
    except BookingService.NoGuardsAvailableError:
        return Response(
            {'error': {'code': 'NO_GUARDS_AVAILABLE',
                'message':
                'No guards available in your area right now.'}},
            status=status.HTTP_404_NOT_FOUND
        )

def retrieve(self, request, pk=None):
    """GET /api/bookings/{id}/"""
    booking = self.get_object()
    return Response({'data':
        BookingDetailSerializer(booking).data})

@action(detail=True, methods=['post'])
def cancel(self, request, pk=None):
    """POST /api/bookings/{id}/cancel/"""
    booking = self.get_object()
    serializer = BookingCancelSerializer(data=request.data)
    serializer.is_valid(raise_exception=True)
    try:
        BookingService.cancel_booking(
            booking=booking,
            cancelled_by=request.user,
            reason=serializer.validated_data.get('reason', '')
        )
        return Response({'data': {'message': 'Booking
        cancelled successfully.'}})
    except TransitionNotAllowed:
        return Response(
            {'error': {'code': 'INVALID_STATE',
                'message': f'Cannot cancel a booking
                in {booking.status} state.'}},
            status=status.HTTP_400_BAD_REQUEST
        )

```

```

    )

    @action(detail=True, methods=['post'], url_path='generate-
        start-otp')
    def generate_start_otp(self, request, pk=None):
        """POST /api/bookings/{id}/generate-start-otp/"""
        booking = self.get_object()
        if booking.user != request.user:
            return Response({'error': {'code':
                'PERMISSION_DENIED'}}), status=403)
        if booking.status != 'ARRIVED':
            return Response(
                {'error': {'code': 'INVALID_STATE',
                    'message': 'Guard must have arrived
                before generating start OTP.'}},
                status=400
            )
        otp = booking.generate_start_otp()
        return Response({'data': {'otp': otp, 'expires_in': 300}})

    @action(detail=True, methods=['post'], url_path='verify-start-
        otp')
    def verify_start_otp(self, request, pk=None):
        """POST /api/bookings/{id}/verify-start-otp/ - Guard
        verifies OTP to start session."""
        booking = self.get_object()
        otp = request.data.get('otp')
        if not booking.verify_start_otp(otp):
            return Response(
                {'error': {'code': 'INVALID_OTP', 'message':
                'Incorrect OTP.'}},
                status=400
            )
        BookingService.start_session(booking)
        return Response({'data': {'message': 'Session started.',
            'started_at': booking.session_started_at}})

```

GuardNearbyView

apps/guards/views.py (partial)

```

from django.contrib.gis.geos import Point
from django.contrib.gis.measure import D
from django.contrib.gis.db.models.functions import Distance
from rest_framework.views import APIView
from rest_framework.response import Response
from .models import GuardProfile
from .serializers import NearbyGuardSerializer

```

```

class NearbyGuardsView(APIView):
    """

```

```

GET /api/guards/nearby/?
    lat=12.97&lng=77.59&type=UNARMED&radius=5
Returns available guards within radius, sorted by distance.
"""
def get(self, request):
    lat = float(request.query_params.get('lat', 0))
    lng = float(request.query_params.get('lng', 0))
    guard_type = request.query_params.get('type', 'UNARMED')
    radius_km = float(request.query_params.get('radius', 5))

    user_location = Point(lng, lat, srid=4326)

    guards = GuardProfile.objects.filter(
        is_online=True,
        verification_status='ACTIVE',
        guard_type=guard_type,
        current_location__distance_lte=(user_location,
D(km=radius_km))
    ).annotate(
        distance=Distance('current_location', user_location)
    ).order_by('distance', '-average_rating')[:20]

    return Response({'data': NearbyGuardSerializer(guards,
many=True).data})

```